

```
import torch
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from datasets import load_dataset
from transformers import BertTokenizer, BertForSequenceClassification, Trainer, TrainingArguments
from sklearn.metrics import accuracy_score, precision_recall_fscore_support, confusion_matrix
```

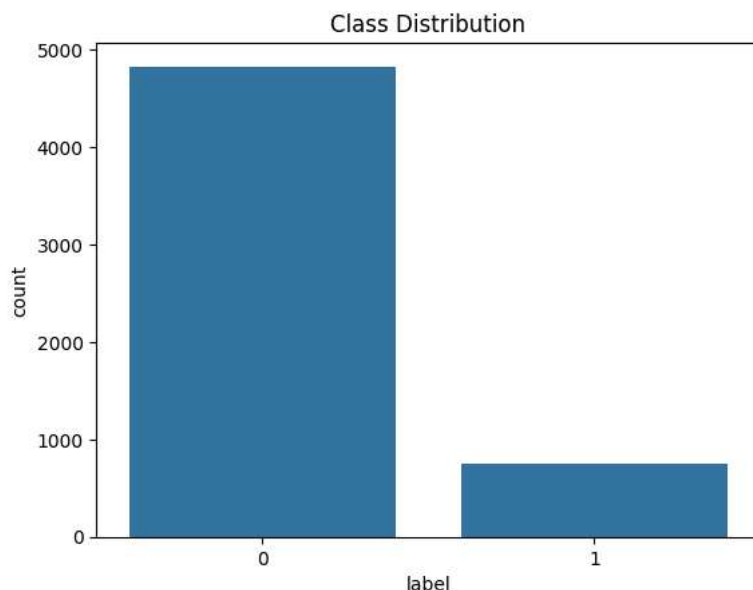
```
# Load dataset
dataset = load_dataset("sms_spam")

# Show sample records
print(dataset['train'][0])
print(dataset['train'][1])

# Convert to pandas for exploration
df = pd.DataFrame(dataset['train'])
print(df.head())

# Class distribution
sns.countplot(x=df['label'])
plt.title("Class Distribution")
plt.show()
```

/usr/local/lib/python3.12/dist-packages/huggingface_hub/utils/_auth.py:94: UserWarning:
The secret `HF\_TOKEN` does not exist in your Colab secrets.
To authenticate with the Hugging Face Hub, create a token in your settings tab (<https://huggingface.co/settings/tokens>), set it
You will be able to reuse this secret in all of your notebooks.
Please note that authentication is recommended but still optional to access public models or datasets.
warnings.warn(
README.md: 4.98k/? [00:00<00:00, 332kB/s]
Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and faster d
WARNING:huggingface_hub.utils._http:Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to en
plain_text/train-00000-of-00001.parquet: 100% 359k/359k [00:00<00:00, 556kB/s]
Generating train split: 100% 5574/5574 [00:00<00:00, 78324.93 examples/s]
{'sms': 'Go until jurong point, crazy.. Available only in bugis n great world la e buffet... Cine there got amore wat...\n', 'la
{'sms': 'Ok lar... Joking wif u oni...\n', 'label': 0}
sms label
0 Go until jurong point, crazy.. Available only ... 0
1 Ok lar... Joking wif u oni...\n 0
2 Free entry in 2 a wkly comp to win FA Cup fina... 1
3 U dun say so early hor... U c already then say... 0
4 Nah I don't think he goes to usf, he lives aro... 0



```
# Load BERT tokenizer
tokenizer = BertTokenizer.from_pretrained("bert-base-uncased")
```

```
# Tokenization function
def tokenize(batch):
    return tokenizer(batch['sms'], padding='max_length', truncation=True, max_length=128)

# Apply tokenization
dataset = dataset.map(tokenize, batched=True)

# Convert labels to tensor format
dataset = dataset.rename_column("label", "labels")
dataset.set_format(type="torch", columns=['input_ids', 'attention_mask', 'labels'])
```

```
tokenizer_config.json: 100% 48.0/48.0 [00:00<00:00, 3.68kB/s]
vocab.txt: 100% 232k/232k [00:00<00:00, 2.25MB/s]
tokenizer.json: 100% 466k/466k [00:00<00:00, 6.16MB/s]
Map: 100% 5574/5574 [00:01<00:00, 4341.59 examples/s]
```

```
# Split dataset into train and test sets
split_dataset = dataset['train'].train_test_split(test_size=0.2, seed=42)
train_dataset = split_dataset['train']
test_dataset = split_dataset['test']

print(f"Train size: {len(train_dataset)}, Test size: {len(test_dataset)}")
```

```
Train size: 4459, Test size: 1115
```

```
# Load BERT model for classification
model = BertForSequenceClassification.from_pretrained("bert-base-uncased", num_labels=2)
```

```
config.json: 100% 570/570 [00:00<00:00, 53.2kB/s]
model.safetensors: 100% 440M/440M [01:32<00:00, 15.9MB/s]
Loading weights: 100% 199/199 [00:00<00:00, 495.28it/s, Materializing param=bert.pooler.dense.weight]
```

```
BertForSequenceClassification LOAD REPORT from: bert-base-uncased
```

Key	Status
cls.predictions.transform.LayerNorm.weight	UNEXPECTED
cls.predictions.transform.dense.weight	UNEXPECTED
cls.seq_relationship.bias	UNEXPECTED
cls.predictions.bias	UNEXPECTED
cls.seq_relationship.weight	UNEXPECTED
cls.predictions.transform.LayerNorm.bias	UNEXPECTED
cls.predictions.transform.dense.bias	UNEXPECTED
classifier.bias	MISSING
classifier.weight	MISSING

Notes:

- UNEXPECTED :can be ignored when loading from different task/architecture; not ok if you expect identical arch.
- MISSING :those params were newly initialized because missing from the checkpoint. Consider training on your downstream task.

```
# Define metrics
def compute_metrics(pred):
    labels = pred.label_ids
    preds = pred.predictions.argmax(-1)
    precision, recall, f1, _ = precision_recall_fscore_support(labels, preds, average='binary')
    acc = accuracy_score(labels, preds)
    return {"accuracy": acc, "precision": precision, "recall": recall, "f1": f1}
```

```
# Training arguments
training_args = TrainingArguments(
    output_dir="./results",
    eval_strategy="epoch",
    learning_rate=2e-5,
    per_device_train_batch_size=16,
    per_device_eval_batch_size=16,
    num_train_epochs=3,
    weight_decay=0.01,
    logging_dir="./logs",
    logging_steps=10
)
```

```
`logging_dir` is deprecated and will be removed in v5.2. Please set `TensorBoardLoggingDir` instead.
```

◆ Gemini

```
from transformers import DistilBertTokenizer, DistilBertForSequenceClassification

# Use DistilBERT instead of BERT
tokenizer = DistilBertTokenizer.from_pretrained("distilbert-base-uncased")
model = DistilBertForSequenceClassification.from_pretrained("distilbert-base-uncased", num_labels=2)

# (Optional) reduce dataset size for speed
small_train = train_dataset.shuffle(seed=42).select(range(2000)) # only 2000 samples
small_test = test_dataset.shuffle(seed=42).select(range(500)) # only 500 samples

# Training arguments (lighter config)
training_args = TrainingArguments(
    output_dir="./results",
    eval_strategy="epoch",
    learning_rate=3e-5,
    per_device_train_batch_size=8, # smaller batch size
    per_device_eval_batch_size=8,
    num_train_epochs=1, # only 1 epoch
    weight_decay=0.01,
    logging_dir="./logs",
    logging_steps=10
)

trainer = Trainer(
    model=model,
    args=training_args,
    train_dataset=small_train,
    eval_dataset=small_test,
    compute_metrics=compute_metrics
)

trainer.train()
```

Loading weights: 100% 100/100 [00:00<00:00, 367.16it/s, Materializing param=distilbert.transformer.layer.5.sa_layer_norm.weight]

DistilBertForSequenceClassification LOAD REPORT from: distilbert-base-uncased

Key	Status
-----+-----	-----+-----
vocab_layer_norm.weight	UNEXPECTED
vocab_transform.weight	UNEXPECTED
vocab_projector.bias	UNEXPECTED
vocab_transform.bias	UNEXPECTED
vocab_layer_norm.bias	UNEXPECTED
pre_classifier.bias	MISSING
pre_classifier.weight	MISSING
classifier.bias	MISSING
classifier.weight	MISSING

Notes:

- UNEXPECTED :can be ignored when loading from different task/architecture; not ok if you expect identical arch.
- MISSING :those params were newly initialized because missing from the checkpoint. Consider training on your downstream task.

`logging\_dir` is deprecated and will be removed in v5.2. Please set `TENSORBOARD\_LOGGING\_DIR` instead.

/usr/local/lib/python3.12/dist-packages/torch/utils/data/dataloader.py:775: UserWarning: 'pin_memory' argument is set as true but not supported by DataLoaderIterator. Ignoring it.
super().__init__(loader)

[37/250 02:45 < 16:49, 0.21 it/s, Epoch 0.14/1]

Epoch Training Loss Validation Loss